Em rà soát một số lỗi sau nhé:

Lỗi 1: Toàn bộ 516 ảnh IDRiD bị gom thành một bệnh nhân

`merge-dataset.ipynb` · hàm `parse_patient_and_side`

MÃ HIỆN TẠI

```
clean_id = img_str.replace("_left","").replace("_right","")...   # "IDRiD_001" ->
"idrid_001"
patient_id = clean_id.rsplit("_", 1)[0] if "_" in clean_id else clean_id
#                    ^^^^^^^^^^^^^^^^^ lấy phần TRƯỚC gạch dưới cuối -> "idrid"
```

Kết quả kiểm chứng bằng cách chạy hàm:

```
IDRiD_001  ->  patient_id = "idrid"
IDRiD_002  ->  patient_id = "idrid"
IDRiD_413  ->  patient_id = "idrid"

Số patient_id duy nhất trên 516 ảnh IDRiD = 1
```

Hậu quả: GroupShuffleSplit coi cả 516 ảnh là một nhóm duy nhất, nên **toàn bộ IDRiD rơi hết vào một split**. Kiểm tra lại log chạy sẽ thấy IDRiD chỉ xuất hiện ở train hoặc test, không có ở cả ba.

Lỗi 2: patient_id đụng nhau giữa các dataset

Hàm parse không gắn tên dataset vào patient_id, nên các ID trùng số bị gộp chung:

```
EyePACS  "10_left"  ->  patient_id = "10"
DeepDRiD "10_l1"    ->  patient_id = "10"   <-- cùng nhóm với EyePACS

Kiểm tra va chạm EyePACS_10 vs DeepDRiD_10 : True
```

Lỗi này không gây leakage (nó gom quá tay nên an toàn hơn, không nguy hiểm hơn) nhưng nó phá tỷ lệ 70/15/15 và làm con số thống kê số bệnh nhân trở nên vô nghĩa.

Cách sửa cả lỗi 1 và lỗi 2

```
def parse_patient_and_side(image_id: str, dataset_key: str) -> tuple[str, str]:
    img_str = str(image_id).lower()
    k = dataset_key.lower()

    if "idrid" in k:
        # IDRiD_001 -> "001"  (lấy phần SAU gạch dưới, không phải trước)
        base = img_str.replace(".jpg","").replace(".png","")
        pid  = base.split("_")[-1]

    elif "eyepacs" in k or "deepdrid" in k:
        pid = img_str.split("_")[0]

    elif "ddr" in k:
        # DDR dùng gạch NGANG: 007-0004-000 -> nhóm theo trường giữa
        parts = img_str.replace(".jpg","").split("-")
        pid = "-".join(parts[:2]) if len(parts) >= 2 else img_str

    elif "messidor" in k:
        # Messidor-2 CÓ cặp 2 mắt: 20051020_43808_0100_PP
```

```
        # -> nhóm theo trường ngày khám (trường đầu)
        pid = img_str.split("_")[0]

    else:                       # APTOS: 1 ảnh = 1 bệnh nhân
        pid = img_str

    side = "left" if "left" in img_str or "_l" in img_str else \
           "right" if "right" in img_str or "_r" in img_str else "unknown"

    return f"{dataset_key}::{pid}", side     # <-- BẮT BUỘC có prefix dataset
```

Kiểm chứng bắt buộc sau khi sửa

In ra bảng số bệnh nhân duy nhất theo từng dataset. IDRiD phải ra ~400+ chứ không phải 1:

`df.groupby("dataset_name")["patient_id"].nunique()`

Lỗi 3: Cân bằng lớp trước khi chia split (nghiêm trọng)

`merge-dataset.ipynb` ở cell thực thi cuối

```
# THỨ TỰ HIỆN TẠI
master_balanced_df = create_balanced_master_pool(master_df, target_range=(1500,2500))
train_raw_df, val_raw_df, test_raw_df =
split_master_pool_patient_level(master_balanced_df)
#
^^^^^^^^^^^^^^^^^
#                           cân bằng ĐÃ chạy trước -> val/test cũng bị cân bằng
```

Mâu thuẫn nội tại: markdown ở cell 9 ghi *"Validation & Test sets: Left untouched in raw distribution"* nhưng code đang làm điều ngược lại.

Vấn đề	Hệ quả
Test set bị cân bằng nhân tạo (mỗi grade ~20%)	Thực tế lâm sàng grade 0 chiếm 70–75% sàng lọc. **Referable DR specificity bị thổi phồng** vì số ca âm tính bị cắt đi hàng chục lần.
QWK tính trên phân bố nhân tạo	**Không so sánh được với bất kỳ bài báo nào**, kể cả bài Hashmi dùng làm đối chứng.
53.977 ảnh → còn ~10.000	Vứt bỏ **80% dữ liệu**, gần như toàn bộ EyePACS. Mất đi lợi thế quy mô của bài.
Không trả lời được câu hỏi reviewer	"Prevalence của test set là bao nhiêu?", hiện không có câu trả lời hợp lệ.

Cách sửa

```
# THỨ TỰ ĐÚNG: chia trước -> chỉ cân bằng trên TRAIN

train_raw, val_raw, test_raw = split_master_pool_patient_level(master_df)   # dùng
master_df ĐẦY ĐỦ

train_bal = create_balanced_master_pool(train_raw, target_range=(1500,2500))  # chỉ
train
val_final  = val_raw      # GIỮ NGUYÊN phân bố tự nhiên
```

```
test_final = test_raw     # GIỮ NGUYÊN phân bố tự nhiên

# In prevalence để đưa vào bài báo
print(test_final["diagnosis"].value_counts(normalize=True).sort_index())
```

Lỗi phụ trong cùng hàm

Đoạn top-up cuối hàm 'create_balanced_master_pool' so sánh index sai:

```
selected_subset = pd.concat(sampled_datasets, ignore_index=True)   # index đã reset về
0..N-1
...
remaining =
grade_subset_existing[~grade_subset_existing.index.isin(selected_subset.index)]
#
^^^^^^^^^^^^^^^^^^^^^
#           so index đã reset với index gốc -> lọc sai -> có thể thêm TRÙNG ảnh
```

Sửa: giữ một cột định danh ổn định (ví dụ `raw_full_path`) và lọc theo cột đó thay vì theo index.

Lỗi 4: Mask ground-truth của XAI được tạo từ chính Grad-CAM

`main_model.ipynb` ở hàm `compute_quantitative_xai`

```
cam, _ = grad_cam.generate_cam(g_img, l_patch)
cam_resized = cv2.resize(cam, (300, 300))

# 2. Extract ground-truth lesion segmentation mask (e.g. from IDRiD benchmark)
_, mask_gt = cv2.threshold(np.uint8(cam_resized * 255), 150, 255, cv2.THRESH_BINARY)
#                                  ^^^^^^^^^^^ mask "ground truth" tạo TỪ CHÍNH CAM
mask_gt = mask_gt / 255.0

cam_binary = (cam_resized >= 0.5).astype(np.float32)
iou = intersection / (union + 1e-7)      # so CAM với mask-của-CAM
```

Đây là phép tự so sánh không phải kiểm định. IoU và Pointing Game sẽ luôn ra rất cao một cách tự động, và con số đó không mang bất kỳ ý nghĩa nào.

Cách sửa:

1. Vô hiệu hoá hàm này ngay (comment lại cell 72 phần XAI) cho tới khi có mask thật.
2. Tải mask thật từ IDRiD: thư mục `Segmentation Groundtruths/` gồm 4 loại tổn thương (MA / HE / EX / SE) cho 81 ảnh.
3. Hợp nhất 4 loại mask thành một mask nhị phân "có tổn thương", resize về cùng kích thước với CAM.
4. Đây chính là lý do IDRiD không được dùng làm test set, nó là dataset duy nhất có mask pixel-level.

Lỗi 5: calculate_monotonicity_violations luôn trả về 0.0

Đây là metric duy nhất chứng minh giả thuyết H3 của em và nó đang hỏng.

```
cum_probs = np.cumsum(probs, axis=1)          # probs là xác suất KHÔNG ÂM
```

```
diffs      = np.diff(cum_probs, axis=1)        # -> cumsum LUÔN không giảm
violations = np.any(diffs < -1e-4, axis=1)     # -> diffs LUÔN >= 0 -> LUÔN False
```

Kiểm chứng trên 10.000 mẫu ngẫu nhiên cho mỗi head:

```
Softmax ngẫu nhiên : violation rate = 0.0000   (min diff = 0.000012)
CORAL              : violation rate = 0.0000   (min diff = 0.000002)
CORN               : violation rate = 0.0000   (min diff = 0.000020)
CLM                : violation rate = 0.0000   (min diff = 0.000007)
```

Nghĩa là con số "0.0% violations" em đặt làm mốc cho H3 là **artifact của bug, không phải kết quả thí nghiệm.**

Cách sửa

Phải đo trên xác suất tích lũy P(y > k) của CORAL và phải đo TRƯỚC bước 'F.relu' vì relu đang che mất chính cái vi phạm cần đếm.

```
def rank_monotonicity_violations(rank_probs: np.ndarray) -> float:
    """
    rank_probs: (N, C-1) = P(y > k), k = 0..C-2
    Ràng buộc thứ hạng: P(y>0) >= P(y>1) >= ... >= P(y>C-2)   (KHÔNG TĂNG)
    """
    diffs = np.diff(rank_probs, axis=1)        # phải <= 0
    return float(np.mean(np.any(diffs > 1e-4, axis=1)))
```

- **CORAL** là head duy nhất có thể vi phạm (các ngưỡng bias học độc lập).
- **CORN** dùng cumprod nên đảm bảo về mặt toán học.
- **CLM** dùng softplus tích lũy nên cũng đảm bảo.

Chính vì vậy kết quả kỳ vọng của H3 phải là: CORAL có tỷ lệ vi phạm > 0, còn CORN và CLM bằng 0. Nếu tất cả đều bằng 0 thì metric vẫn đang hỏng.

Lỗi 6: Em cần thiết lập lại để có kết quả thực nhé

`main_model_head_ablation.ipynb` ở cell 0, bảng "Research Hypotheses Verification Mapping"

Giả thuyết	"Target Benchmark" đang ghi trong file	Tình trạng
H1	+36.9% over Global Baseline	**Chưa chạy thí nghiệm nào**
H2	+18.2% Grade 3 Recall	**Chưa chạy thí nghiệm nào**
H3	0.0% Violations, +0.041 QWK	**Chưa chạy** và metric đang hỏng (Lỗi 5)
H4	≤ 0.052 Generalization Gap	**Chưa có external test set nào**
H5	IoU ≥ 0.45, Pointing ≥ 88%	**Chưa có mask thật** (Lỗi 4)

Đồng thời sửa nhãn sai trong cả hai notebook: tập test hiện tại được gọi là "OOD Test" / "Out-of-Domain" nhưng thực chất là 15% ngẫu nhiên in-domain. Chưa có external hold-out, chưa có LODO trong code. Để nguyên tên này thì mọi kết luận về domain shift (tức toàn bộ câu chuyện của bài) đều sai nhãn.

Nhánh Local MIL chưa đủ để chứng minh H1

`extract_mil_patches` hiện đang chia lưới 4×4 đều trên bounding box

Tính toán độ bao phủ thực tế:

Kích thước ảnh gốc	K=16 patch phủ được	Khoảng cách giữa 2 patch cạnh nhau	Vùng bỏ trống
1500 × 1500	14,8% vùng võng mạc	457 px	329 px
2000 × 2000	8,3% vùng võng mạc	624 px	496 px
3000 × 3000	3,7% vùng võng mạc	957 px	829 px
4000 × 4000	2,1% vùng võng mạc	1290 px	1162 px

Một vi phình mạch kích thước 20–60 px nằm trong khoảng trống 500–800 px đó thì mô hình không bao giờ nhìn thấy. **H1 không thể chứng minh được với thiết kế này.**

- Thêm nữa là chưa lọc FOV. Ảnh đáy mắt là hình tròn nội tiếp khung vuông nên ít nhất 4 patch ở 4 góc bounding box gần như toàn nền đen.
- Đã thống nhất chuyển sang sliding-window MIL nhưng code vẫn là grid 4×4 cũ. Em cần tăng K, giảm stride, lọc patch theo tỷ lệ pixel nằm trong FOV và lấy mẫu ngẫu nhiên M = 32–64 patch mỗi epoch khi train (quét toàn bộ lưới khi inference).

H2 chưa có thành phần kiến trúc nào để kiểm chứng

Markdown gán H2 cho "Global Branch + CBAM" nhưng CBAM spatial chỉ là một conv 7×7 trên bản đồ đã pooled, đúng cơ chế đã được phân tích là không giải được quy tắc 4-2-1 của grade 3. Hiện chưa có Non-local, chưa có Quadrant Tokens. **Nếu chạy thí nghiệm bây giờ thì H2 sẽ ra kết quả âm tính vì lý do kiến trúc chứ không phải vì giả thuyết sai.**

Các lỗi kỹ thuật cần sửa

Vị trí	Vấn đề	Cách sửa
`FundusDualBranchDataset.__getitem__`	`row["native_path"].split("\\")[-1]` hard-code dấu phân cách Windows → vỡ trên Linux/Colab	Dùng `Path(row["native_path"]).name`
`fusion_layer` + `train_loader`	`nn.BatchNorm1d(1024)` với `batch_size=8`: thống kê nhiễu và batch cuối có thể còn 1 mẫu → **crash** ở train mode	Thêm `drop_last=True` cho train_loader, hoặc đổi sang `LayerNorm`
`LocalMILBranch.forward`	Chỉ dùng `torch.max`, đúng cho grade 1 nhưng **không đếm được** nên grade 3 không có cơ chế nào hỗ trợ	Dual pooling: `concat(max_pool, gated_attention_pool)`
Global branch	Vẫn ở 300×300, grade 1 gần như vô hình ở độ phân giải này	Nâng lên 512 tối thiểu
`generate_feature_cache` (cell 52)	Gọi `model.extractor.extract_features(...)`, `DRFullModel` không có `.extractor`, `DualBranchMILModel`	Xóa hẳn cell 52. Bản đúng đã có ở cell 64.

Vị trí	Vấn đề	Cách sửa
	không có `.extract_features` → **AttributeError**	
`CumulativeLinkModelQWKHead`	`qwk_weight=1.2` bật ngay từ epoch 1 → đúng vấn đề stagnation mà bài Vargas cảnh báo	Cho λ tăng tuyến tính từ 0 trong 3–5 epoch đầu
`FundusGradCAM.__init__`	`register_full_backward_hook` không bao giờ được gỡ → gọi nhiều lần sẽ chồng hook, rò rỉ bộ nhớ	Lưu handle và gọi `.remove()` hoặc dùng context manager
cell 62	`epochs=3` và `csv_train_path` trỏ vào `sample_50_metadata.csv`	Đây là smoke test. **Kết quả tuyệt đối không đưa vào bài.**

Về câu hỏi của em

Có nên lưu dataset đã merge xuống ổ rồi mới train?

Mình nên lưu nhưng nguyên nhân 1,5 giờ không nằm ở chỗ em nghĩ

Gốc rễ là em đang ghi file ảnh vào thư mục theo split:

```
save_dir = os.path.join(NATIVE_PROCESSED_DIR, split_name, str(row["diagnosis"]))
out_name = f"{split_name}_g{row['diagnosis']}_{image_id}.png"
#            ^^^^^^^^^^^^^^^^^^^^^^^^ tên file và thư mục DÍNH vào split và nhãn
```

Vì file ảnh dính chặt vào split, mỗi lần đổi fold là phải ghi lại toàn bộ ảnh. Đó là lý do LODO tốn 1,5 giờ × 6 vòng.

Kiến trúc đúng: tách làm hai tầng độc lập

```
# ---- TẦNG 1: tiền xử lý, chạy ĐÚNG MỘT LẦN trong đời dự án ----
# Lưu theo dataset + image_id. KHÔNG theo split. KHÔNG theo grade.
out = f"{PROC}/{dataset_key}/{image_id}.jpg"
if os.path.exists(out):        # resume được nếu chạy đứt giữa chừng
    continue

# ---- TẦNG 2: split chỉ là file CSV ----
folds/fold_lodo_messidor2.csv    # cột: image_path, diagnosis, patient_id, dataset,
split
folds/fold_lodo_idrid.csv
folds/fold_internal_70_15_15.csv
# Sinh 6 file CSV này mất VÀI GIÂY, vì chỉ gán nhãn split trên DataFrame.
```

Ba tối ưu thêm để 1,5 giờ xuống còn khoảng 10 phút:

1. Dùng `joblib.Parallel(n_jobs=8)` cho vòng lặp đọc–crop–ghi. Hiện đang chạy tuần tự bằng `df.iterrows()`
2. Lưu JPEG quality 95 thay vì PNG. PNG encode ảnh 3000×2000 rất tốn mà ảnh đáy mắt gốc vốn đã là JPEG.
3. Bỏ `.iterrows()` dùng list of dict rồi tạo DataFrame một lần ở cuối.

Load backbone rồi gắn 5 head có hợp lý không?

Hợp lý về kỹ thuật (tiết kiệm khoảng 5× compute) nhưng không hợp lệ về mặt khoa học cho bảng kết quả cuối cùng.

Nhìn lại thứ tự thực thi trong main_model.ipynb:

```
# cell 62
main_model = build_model(head_type="CLM_QWK", ...)   # train backbone CÙNG head CLM
main_trainer.fit(...)

# cell 63-64
backbone = main_model.backbone
backbone.eval(); freeze
extract_and_cache_latents(backbone, ...)             # cache latent

# -> head_ablation.ipynb: so 5 head trên latent này
```

Backbone đã được huấn luyện cùng CLM_QWK nên latent 512 chiều đó đã tối ưu riêng cho CLM. Em đem latent đó đi so 5 head rồi kết luận CLM tốt nhất, đó là **thiên vị có hệ thống cho đúng cái head em muốn chứng minh.** Reviewer có thể sẽ bắt chỗ này.

Mục đích sử dụng	Dùng latent cache?	Ghi chú
Sàng lọc nhanh: tuning λ, learning rate, chọn link function	**Được**	Giữ nguyên cách làm hiện tại, rất hiệu quả
Bảng kết quả cuối trong bài báo	**Không**	**Bắt buộc train end-to-end riêng cho từng head, 5 lần.** Không có đường tắt
Phương án B nếu máy không kham nổi	Được, có điều kiện	Train backbone bằng **Softmax_CE** (head trung lập, không phải head đề xuất) rồi mới đóng băng và ghi rõ trong Limitations

Lỗi phụ trong khâu cache latent

Latent tập train đang trích từ `train_loader` vốn có augmentation ngẫu nhiên → mỗi ảnh chỉ có MỘT latent của một lần augment duy nhất. Head sẽ overfit rất nhanh và latent không đại diện. Cách sửa: em tạo riêng một loader cho tập train với `split="val"` (tắt augmentation) chỉ để trích latent.